

ComplexGitSync 0002.58– User Guide

Nicolas Flipó

September 16, 2026

Contents

1	User Guide	2
2	CLI Overview	2
3	Lifecycle Contract	2
4	Commands	3
4.1	initialise	3
4.2	bootstrap	4
4.3	discover	5
4.4	configure	6
4.5	create-cgs	6
4.6	clean-init	6
4.7	purge	6
4.8	pull	6
4.9	pull-force	7
4.10	clone	7
4.11	checkout	8
4.12	add	8
4.13	rm	9
4.14	commit	9
4.15	push	9
4.16	freeze-release	9
4.17	freeze	10
4.18	import-submodules	10
4.19	init-from-submodules	11
4.20	verify	12
4.21	launch-release	12
4.22	view-tree	12
4.23	What cgitssync promises a script	13
4.24	Where a command works: the default workspace	13
4.25	status	14
4.26	merge	15
4.27	pull -private	16
5	Document Formats	16
5.1	Local Authoring Spec (.cgs)	17
5.2	Branches in a .cgs	18
5.2.1	What private means	19
5.2.2	Read-only and writable configuration repositories	19

5.2.3	Which repositories each command touches	21
5.2.4	The two meanings of “force”	21
5.3	Repository Placement and Nested Discovery	21
5.4	Git Tree State Snapshot (.gts)	22
5.5	Local Git Register and Sync Ledger (.lgr)	23
6	Worked Examples: From CGSi11 to cawaqsviz	23
6.1	Level 1 — CGSi11: the minimal synthetic topology	24
6.2	Level 2 — cawaqs: the same minimal style, at scale	25
6.3	Level 3 — cawaqsviz: three ways to the same .cgs	25
6.3.1	3a — Write it by hand	25
6.3.2	3b — Derive it with discover (autodiscover mode)	26
6.3.3	3c — Migrate it with import-submodules	26
6.3.4	A shared caveat across all three	27
7	Configuration and Environment	27
8	Error Reference	27
9	Troubleshooting	27

1 User Guide

This chapter is the authoritative reference for every user-facing command, configuration option, document format, and error condition exposed by ComplexGitSync.

2 CLI Overview

The single entry-point is the `cgitsync` command.

cgitsync is a Pixi task, not a globally installed executable. This project is developed and run exclusively through Pixi (`pixi install` once, per checkout) — there is no `pip install` that puts a bare `cgitsync` on `$PATH`. Every invocation shown in this chapter as `pixi run cgitsync ...` must be typed exactly that way; a bare `cgitsync ...` will not be found by the shell.

```
$ pixi run cgitsync <command> [options]
```

Running `cgitsync` without a command prints a help summary and exits with code 0.

ComplexGitSync stores no credentials of its own and has no private-repository authentication story: every git operation — `clone`, `pull`, `push`, and the remote lookups behind `checkout`’s branch/tag fallback — runs against whatever the ambient environment already has cached (`ssh-agent`, an HTTPS credential helper, a stored personal-access-token). When that ambient credential is missing or stale, the operation fails immediately with a normal, catchable error rather than sitting silently on a terminal or GUI credential prompt that will never arrive — every git subprocess ComplexGitSync runs sets `GIT_TERMINAL_PROMPT=0` and a no-op `GIT_ASKPASS` for exactly this reason. Fix the ambient credential (switch the remote to `ssh`, refresh the token) and re-run the command; nothing needs to be undone first.

3 Lifecycle Contract

This guide follows the simplified canonical lifecycle:

1. `initialise(.cgs)` → clone all repos → **READY** (*new project*)
`initialise(.gts)` → restore from snapshot → **READY** (*existing project*)
2. `pull(.cgs|.gts)` → resync an existing tree
3. `checkout(.gts)` / `add` / `commit` / `push` / `tag` → git operations on the tree
4. `freeze` → emit the next `.gts` id and update `<Project_name>.lgr`

The project-local `.lgr` register and sync ledger are both active (T24, T32). Every synchronisation operation is automatically recorded as an immutable DAG event in the `[[ledger]]` section of the `.lgr` file.

4 Commands

The CLI exposes three command groups once a `GitTree` is **READY**. The minimalist level is `initialise`, `bootstrap`, `clean-init`, `freeze-release`, `freeze-release-force`, `status`, `view-tree`, and `launch-release`. The expert level exposes the individual operations: `branch`, `checkout`, `purge`, `validate`, `clone`, `pull`, `pull-force`, `add`, `commit`, `push`, `tag`, `freeze`, `import-submodules`, `init-from-submodules`, and `verify`. The configuration level, `discover`, `configure` and `create-cgs`, authors a `.cgs` specification without requiring existing state. Redundant inspection aliases such as `print`, `view-operation`, and `validate-topology` are not part of the public CLI surface.

4.1 initialise

```
$ pixi run cgitssync initialise <spec.cgs> [--output-path DIR] [--commit-
gitignore]
    [--force-gitignore-sync] [--git-user-name NAME] [--git-user-email
    EMAIL]
    [--force-protocol {ssh,https}] [--force-reclone]
$ pixi run cgitssync initialise <snapshot.gts>
$ pixi run cgitssync initialise --project NAME --repo PROVIDER:OWNER/REPO
    [--repo ...] [--output-path DIR]
```

Unified initialisation entry point. Dispatches on file extension:

- `.cgs` source (clone mode): clones the full repository tree. The optional `-output-path` controls `CGSPATH`; `CGSHOME` is derived as `CGSPATH/<project-name>`. When omitted, `CGSPATH` defaults to `../...`. On success, ends in **READY**. If clone setup fails because stale generated state is still present, the CLI prints `Try clean-init method`.
- `.gts` source (restore mode): loads a saved snapshot directly. On success, ends in the lifecycle state recorded in the snapshot.
- Direct CLI authoring: requires `-project` and at least one repeatable `-repo`. It is mutually exclusive with a source file. The CLI collects values only and calls `ComplexGitSyncClient.configure()`; that reusable Python API delegates parsing, normalization, and validation to `cgs_format.py` and produces the same canonical `CgsDocument` as an equivalent `.cgs` file.

All initialization paths end in a **READY** tree or fail explicitly.

initialise re-clones your dependencies. Only the root repository at CGSHOME is kept as it is. Every repository below it whose directory already holds files is **deleted and cloned again**. The old `.git` goes with it, so nothing in that directory can be recovered once the run has passed that point.

Before deleting anything, **initialise** inspects each destination and stops the whole run when one of them holds work that exists nowhere else: uncommitted changes, commits that have not been pushed to the branch's upstream, or a branch with no upstream at all. It names every repository that blocked it, in one message, and deletes none of them. The two ways forward are to commit and push that work, or to pass `-force-reclone` and delete it deliberately.

A destination that is not a Git checkout at all — what a clone interrupted halfway leaves behind — is still cleared without any flag. That is the case the deletion was written for, and it still works.

Before a `.cgs` source is confirmed ready, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled parent-first and has its `.gitignore` updated with the relative path of each immediate child — nested repositories are plain independent clones rather than gitlinks, so without this a parent's `git status/git add` would otherwise see a child's working tree as ordinary untracked content. This is logged as the `GT-GITIGNORE` phase, after `GT-CLONE`. If the safe pull for one of these repositories fails, **initialise** raises immediately and nothing is written; pass `-force-gitignore-sync` to instead fall back to a pull-force recovery for that one repository (never a force-push). By default nothing is staged, committed, or pushed by this step — it only writes the file and prints what changed. Pass `-commit-gitignore` to explicitly approve staging (`.gitignore` alone), committing (with a message listing exactly which children were added), and pushing each changed repository. The commit identity defaults to local git config; `-git-user-name/-git-user-email` override it and persist the override to `CGSHOME/.cgitsync/master.toml` via `MasterConfig`, so later invocations on the same workspace pick it up without repeating the flags.

The root's own `.gitignore` additionally always gains `.cgitsync/` (the generated runtime-state directory — snapshots, register, run logs, never part of the `.cgs/.gts` project spec) and its `<project-name>.lgr` register file, whether or not the tree has any nested repositories at all — that state is written under the root regardless of tree shape, so it is excluded the same idempotent way, via the same underlying write `sync_gitignore()` already uses for child mount points.

`-force-protocol {ssh,https}` is an expert option, typically used systematically in CI rather than for everyday development. It overrides `access_protocol` in memory for every repository this run clones — including ones discovered later from a nested `.cgs` living in a different, separately-cloned repository — so no `.cgs` file is ever read differently or written to. Forcing `https` is the usual CI case: it removes the dependency on an SSH key or agent being present on the runner, a dependency that otherwise differs between a `push` run and a `pull_request` run (GitHub Actions withholds repository secrets from a pull request opened from a fork). Omitted (the default), each repository follows whatever protocol its own `.cgs` entry declares, exactly as without the flag.

4.2 bootstrap

```
$ pixi run cgitsync bootstrap <spec.cgs> <project_name> [--cgs-path DIR]
  [--force-protocol {ssh,https}]
```

Clones the full repository tree from a `.cgs` source into an isolated CGSHOME, for running `cgitsync` from its own standalone clone rather than nested inside the project tree it manages — one `ComplexGitSync` clone reused across many projects. It delegates to the same `ComplexGitSyncClient.clone_cgs` pipeline as `clone` below, and so shares its branch-fallback and nested-discovery behaviour, but differs from both `clone` and `initialise` in two ways:

- `project_name` is a required positional argument, not inferred from the `.cgs` document. It always forms the final path segment of `CGSHOME`, regardless of the specification’s own project name.
- `CGSPATH` does not default to `../..` relative to the current directory (that default assumes `ComplexGitSync` is cloned inside the tree it manages — the nested-mode case `initialise` is for). When `-cgs-path` is omitted, `CGSPATH` instead defaults to a fresh `$HOME/.cgs/CGS<timestamp>/` directory, created automatically if it does not exist, so a bootstrapped project can never land inside `ComplexGitSync`’s own clone.

`-force-protocol {ssh,https}` behaves exactly as described under `initialise` above, applying here to every repository the run clones — including the root, which `bootstrap` (unlike `initialise`) also clones from scratch rather than assuming already checked out.

4.3 discover

```
$ pixi run cgitsync discover [ROOT] [--write FILE] [--max-depth N]
```

Scans `ROOT` (default: the current directory) for git repositories and drafts a `.cgs` from what is checked out, through the `ComplexGitSyncClient.discover_repos()` facade. This is the entry point for adopting a project that exists on disk but has no `.cgs` describing it yet.

The command is a pure read of the filesystem and of each repository’s git config: nothing is cloned, fetched, staged, or modified, and no network call is made. Without `-write` it only prints what it found, matching the “report first, write only when asked” posture of `-commit-gitignore` and `import-submodules -apply`.

The walk descends with no depth limit by default, treating every directory that contains a `.git` entry as a repository. Pass `-max-depth N` (`ROOT` itself is depth 0) to bound it. It never descends *into* a `.git` directory: for a submodule the real git directory lives at `<parent>/ .git/modules/<name>` while the child’s own `.git` is a file, so walking into it would report the same repository twice.

For each repository, `origin`’s URL is read and converted to the canonical `provider:owner/repository` shorthand through the existing `parse_repo_id()` grammar — this command adds no second parser. `relative_path` is taken straight from the walk rather than from a repository name, so a child mounted at `external/Thing` is recorded there and not at `Thing`. `nested_config` is left unset on every entry: the default `auto` already resolves cleanly whether or not a repository carries its own `*.cgs`.

Repositories with no `origin`, or whose remote URL does not resolve to a registered provider, are reported as warnings and left out of the draft. They are never guessed at: a draft that silently invented an address would be worse than one that states what it could not determine. Always review the generated file, then `validate` it.

A repository found *inside* another repository is drafted as that repository’s child, not the project root’s. Nesting is worked out by comparing paths: every path in a `.cgs` is written from the project root, so a repository whose path lies under another’s sits inside it, and the innermost one wins. The report marks such an entry `inside: <path>` and prints the resulting tree, and `build_registry_from_cgs_document()` reads the drafted file back the same way — the holding repository becomes a `PARENT` and keeps its own `.gitignore` entry for the repository it holds, which is what stops a later `add` from restaging that repository as a submodule.

Only what is *checked out at scan time* can be found. A repository cloned without `-recurse-submodules` leaves its submodule paths as empty directories, and those are correctly not reported; recovering them from git metadata instead is `import-submodules`’ job. The same applies when `-max-depth` is given: if it stops the walk while directories are still left to look into, the report says so in a warning rather than presenting a partial answer as a complete one. Without the flag, the walk cannot stop early — there is nothing to stop it at.

4.4 configure

```
$ pixi run cgitsync configure [--output FILE]
```

Interactively prompts for a project name, default branch, and one or more repositories (provider, owner, name, and per-repository overrides), then writes the resulting `.cgs` file through the same offline `ComplexGitSyncClient.configure()` facade used by `initialise -project/-repo` and `create-cgs`. The provider prompt accepts `github`, `gitlab`, `codeberg`, or `custom`; `custom` additionally prompts for the provider URL. When `-output` is omitted, the CLI prompts for a destination path, defaulting to `<project-name>.cgs`.

4.5 create-cgs

```
$ pixi run cgitsync create-cgs --project CGSil1 \  
  --repo github:flipoyo/ComplexGitSync \  
  --repo codeberg:GX4G/GX4G \  
  --output CGSil1.cgs
```

Builds the canonical `CgsDocument` through the offline format pipeline and serializes concise `.cgs` TOML. It performs no Git or network work.

4.6 clean-init

```
$ pixi run cgitsync clean-init <spec.cgs> [--output-path DIR] [--commit-  
  gitignore]  
  [--force-gitignore-sync] [--git-user-name NAME] [--git-user-email  
    EMAIL]  
  [--force-protocol {ssh,https}]
```

Recovery-oriented initialisation for a `.cgs` source. It follows the same high-level pipeline as `initialise(.cgs)` (including the `.gitignore` lifecycle sync and `-force-protocol` behaviour described there), but inserts a cleanup phase before cloning:

load->expand->validate->purge->clone->gitignore

Use it when an earlier initialisation attempt left partial child checkouts. `CGSHOME/.cgitsync/master.toml` (the persisted Git identity override, if any) is preserved by the purge phase — it is workspace-local configuration, not generated clone state.

4.7 purge

```
$ pixi run cgitsync purge <spec.cgs> [--output-path DIR]
```

Removes generated clone state from the resolved `CGSHOME` without cloning. It deletes child repository directories declared directly under the project root and project-local `*.lgr` files. Nested directories that are not immediate root children, and `.cgitsync/master.toml` (see `clean-init` above), are left untouched by this command.

4.8 pull

```
$ pixi run cgitsync pull [spec.cgs|snapshot.gts] [--commit-gitignore]  
  [--force-gitignore-sync] [--git-user-name NAME] [--git-user-email  
    EMAIL]  
  [--force-protocol {ssh,https}]
```

Resynchronises the project tree. `.cgs` sources reload the source, discover nested configs, then resynchronise the loaded tree. `.gts` sources load the saved registry as the starting point and then run the same pull workflow. In both cases the operation starts at the project root: `root -> parent -> leaf`. Every repository — root, parent, and leaf alike — is a plain independent clone and receives its own `git pull -ff-only`. Use `launch-release` when you want to check out a frozen recorded release state instead of pulling current remotes. If local changes block this safe pull, the CLI prints `You can try cgitssync pull-force` command.

Before pulling your own branch, `pull` fetches *every* branch of each remote. That is what lets `checkout` join a branch somebody else pushed rather than starting a new one of the same name (see `checkout` below). It also repairs a workspace whose clone left a single-branch fetch refspec behind, described under `clone`.

For a `.cgs` source, `pull` also runs the same `.gitignore` lifecycle sync described under `initialise` — including `-commit-gitignore`, `-force-gitignore-sync`, and the `-git-user-name/-git-user-identity` override — once the tree-wide pull completes. A `.gts` source runs no discovery, so there is nothing new for the sync to find. `pull-force` does not run this sync at all; it is a destructive recovery command, not a lifecycle path the sync is wired into.

`-force-protocol {ssh,https}` rewrites every repo’s remote to that protocol before pulling, persisting the change (`git remote set-url`, a no-op when the URL already matches) rather than a one-off override — the switch sticks for every command after this one too, the same way a repo’s protocol at clone time sticks for everything downstream of it. Same meaning as `initialise/bootstrap`’s `-force-protocol`, applied to an already-cloned tree instead of at clone time — useful when a repo was adopted rather than cloned by `ComplexGitSync` (`initialise` never touches an adopted root’s remote) and its remote’s protocol does not match what the ambient environment can actually authenticate with. On a failure that looks like an SSH or HTTPS auth problem, the error gains an actionable hint naming `-force-protocol <the other one>` even without the flag.

The rewrite reads the repository’s *actual current* remote URL and only swaps its scheme — `ssh (user@host:path)` to/from `https/http (scheme://host/path)` — keeping the host and path exactly as they already are. It does *not* rebuild the URL from the repository’s stored identity fields (`gitprovider/project_owner_name/...`): those can be missing or stale for a repository loaded from an older `.gts` snapshot (see the “Formal `.gts` snapshot specification” section of the architecture reference), and rebuilding from a wrong or default-GitHub provider would silently aim the pull at the wrong host. Converting the URL that is already configured never has that failure mode. A remote in neither recognised form (a bare local filesystem path, for instance) cannot have its protocol converted; `-force-protocol` then fails with an explicit error naming the repository, rather than guessing.

4.9 pull-force

```
$ pixi run cgitssync pull-force [spec.cgs|snapshot.gts] [--force-protocol {ssh,https}]
```

Destructively resynchronises the same tree. Every repository — root, parent, and leaf alike — is forced to the selected remote branch, parent-first, with `git fetch`, `git checkout -B <branch> FETCH_HEAD`, and `git clean -fd`. This command can discard local uncommitted changes and untracked files; it never force-pushes. `-force-protocol` behaves exactly as described under `pull` above.

4.10 clone

```
$ pixi run cgitssync clone <spec.cgs> [--target-dir DIR] [--output-path DIR]
```

Clones the full repository tree declared by the `.cgs` file. This CLI command delegates to `ComplexGitSyncClient.clone`. When `-target-dir` is omitted, the project root is derived from the specification. `-output-path` provides the parent directory used to derive the project root.

For each repository, `cgitsync` tries the repo target branch first and falls back to the repo fallback branch when the preferred branch does not exist on the remote. The section “Branches in a `.cgs`” sets out which branch each repository targets and why. Nested `.cgs` files are discovered after parent repositories are cloned, so explicit and automatic nested trees are materialised during the same command.

What the clone leaves behind. A clone still downloads one branch — that is what keeps it quick. What it no longer does is *remember* that restriction: it leaves no single-branch fetch refspec in `.git/config`, so a later `fetch` sees the whole remote rather than the one branch cloned. A workspace cloned before this still carries the narrow refspec; the first `pull` or `push` widens it, with no re-clone needed. `status` deliberately does not: it is a read-only command and never rewrites `.git/config`.

4.11 checkout

```
$ pixi run cgitsync checkout <branch> [--gts <snapshot.gts>] [--ref-kind  
branch|tag]
```

Checks out a branch or tag across the whole tree, parent-first. Loads the READY registry from `-gts`, then propagates the ref, creates missing local branches, and runs `git checkout` in every repo. On success the tree remains READY and a new `.gts` snapshot is written.

A branch this workspace already knows is joined, not recreated. When the repository holds a remote-tracking ref for the branch — because some earlier `pull` downloaded it — `cgitsync` creates the local branch *from that ref*, with tracking set. You land on your colleague’s commits. Only when no such ref exists does it create a new branch where you currently stand.

This is a change of behaviour, and the reason for it is the failure it removes. `cgitsync` used to create the branch at the current HEAD in every case and report READY/ALIGNED — so typing a colleague’s branch name gave you a branch of that name holding commits that had nothing to do with theirs, and nothing in the output said so.

`pull` is what puts those refs on disk: it now fetches every branch of each remote before pulling your own. `checkout` itself still never touches the network, and still works offline — it reads what the last `pull` brought.

To join somebody else’s branch: `pull`, then `checkout <their branch>`.

4.12 add

```
$ pixi run cgitsync add [PATH ...] [--gts <snapshot.gts>] [--dry-run]
```

With no PATH arguments (the default), stages all changes across the tree (`git add -all`), leaf-first. Requires READY; tree stays READY. With `-dry-run`, `cgitsync` prints the leaf-first execution plan and does not mutate repositories.

With one or more PATH arguments, each is resolved (relative to `CGSHOME`, or absolute) to the one repo in the tree that owns it and staged there individually (`git add - <path>`), leaving every other repo untouched. A path outside every repo in the tree raises an error immediately, before anything is staged.

4.13 rm

```
$ pixi run cgit sync rm PATH [PATH ...] [--gts <snapshot.gts>] [--dry-run]
```

Removes one or more tracked files, each resolved the same way as `add`'s targeted form: to the one repo in the tree that owns it, where it is deleted from disk and the removal staged (`git rm - <path>`). A plain tracked file only — a path that resolves to a directory, or that does not exist, raises an error rather than partially applying, and a path outside every repo in the tree raises before anything is removed. Requires `READY`; tree stays `READY`. With `-dry-run`, `cgit sync` prints the plan without mutating repositories.

Distinct from and unrelated to `import-submodules`' internal use of `git rm -cached` (index-only, dropping a gitlink while keeping the child's working tree) — `rm` deletes the file from disk.

4.14 commit

```
$ pixi run cgit sync commit <message> [--gts <snapshot.gts>] [--no-stage] [--dry-run]
```

Commits changes across the tree, leaf-first. Loads the `READY` registry from `-gts`. Repositories with no staged changes after the optional `git add -all` step (suppressed by `-no-stage`) are silently skipped. Requires a `READY` tree; the tree remains `READY` on success. Commit preflight now emits warnings for actionable workspace issues such as dirty worktrees, ahead branches, or stale recorded `commit_sha` values. With `-dry-run`, `cgit sync` prints the planned stage/commit sequence without performing git writes.

The commit message conventionally ends with `CGS#VERSION`.

4.15 push

```
$ pixi run cgit sync push [--gts <snapshot.gts>] [--dry-run] [--force-protocol {ssh,https}]
```

Pushes all repositories to their configured remotes, leaf-first. Loads the `READY` registry from `-gts`. Each repo is pushed to `entry.remote_name` (defaulting to "origin") on its currently resolved branch. Refreshes the stored hash in `GitTree`. Requires a `READY` tree; the tree remains `READY` on success. Push preflight blocks detached HEADs, missing remotes, unresolved merges, and behind/diverged branches; it warns when the local branch is ahead or when the loaded snapshot records an older `commit_sha`. With `-dry-run`, `cgit sync` prints the push plan without mutating any repository. `-force-protocol` behaves exactly as described under `pull` above.

4.16 freeze-release

```
$ pixi run cgit sync freeze-release <name> <message> [--gts <snapshot.gts>] [--dry-run]
  [--force-protocol {ssh,https}]
$ pixi run cgit sync freeze-release-force <name> <message> [--gts <snapshot.gts>] [--dry-run]
  [--force-protocol {ssh,https}]
```

Minimalist release workflow from a `READY` tree. It chains public operations in order: `add` $\rightarrow$ `commit` $\rightarrow$ `pull` $\rightarrow$ `push` $\rightarrow$ `freeze`. The force variant uses `pull-force` instead of `pull`. The release name becomes the shared tag and freeze snapshot label; the message is used for the release commit.

The `pull/pull-force` step is skipped (not attempted) when the current branch has no upstream configured yet — e.g. a branch just created and checked out this same session, never pushed before. There is nothing to pull in that case; `push` (already upstream-aware, adding `-u` automatically the same way plain `git push -u` would) publishes the branch on its own. On any branch that already has an upstream, the workflow is unchanged.

`-force-protocol` is forwarded to the `pull/pull-force` and `push` steps above, behaving exactly as described under `pull`; the remote rewrite it makes persists, so the final `freeze` step's own tag push picks it up too without needing the flag itself.

4.17 freeze

```
$ pixi run cgitssync freeze <name> [--gts <snapshot.gts>] [--dry-run]
```

Performs a release freeze across all repos, leaf-first: stage/commit, create+push the shared release tag, refresh SHA/state, and write a `.gts` snapshot. Loads the READY registry from `-gts`. Requires READY and leaves the tree READY. Freeze preflight checks remote availability, branch alignment, detached HEADs, unresolved merges, and tag absence. Dirty worktrees and stale recorded `commit_sha` values are reported as warnings because freeze will stage, commit, and snapshot them. Freeze snapshots also include a deterministic `freeze_manifest` section that records freeze invariants (immutable snapshot, validated workspace, synchronized tag, `release-name`, ledger checkpoint) and declares `launch_state` as the canonical restore operation. Their immutable snapshot filename includes the release, for example `gts-000001-release-2026.05.gts`, while the ledger id remains `gts-000001`. With `-dry-run`, `cgitssync` prints the planned add/-commit/tag/push graph for the loaded workspace without mutating it.

4.18 import-submodules

```
$ pixi run cgitssync import-submodules REPO_ROOT [--apply] [--recursive]
```

Reads `REPO_ROOT/.gitmodules` and reports (or converts) every declared git submodule to a plain ComplexGitSync nested clone. This is the whole job: turning gitlinks into plain clones on disk. It does not author a `.cgs` — `.gitmodules` never records the root's own identity, and a checkout worth importing already has a `.cgs` (hand-authored) or can get one from `discover` (§1 “discover”), before or after applying.

Without `-apply` (the default), the command performs a *dry run*: it prints each submodule's name, path, URL, and branch, without touching the repository.

`-recursive` (same name and meaning as `git submodule update`'s own flag) also converts any submodule that itself has its own checked-out `.gitmodules`, at any depth — root first, not leaf first: converting a submodule stages changes in its own working tree, and converting a deeper level first would make the shallower level's own preflight check (is the submodule's working tree clean?) see that staged dirt and reject its own conversion; converting the shallower level first avoids this, since `git rm -cached <path>` only ever touches the parent's own index, never the submodule's working tree. A submodule path that was never checked out (`git submodule update -init` not run for it) has no `.git` to read a nested `.gitmodules` from and is invisible either way — the same ceiling `discover` already has for itself. Without `-recursive`, behavior is unchanged: exactly `REPO_ROOT/.gitmodules`, one level.

The report lists every level together. Each entry's `path` is counted from `REPO_ROOT`, and `declared in` names the `.gitmodules` file it was read from. A submodule's own `.gitmodules` path means nothing on its own once more than one repository is involved — in `cawaqsviz`, two of the three submodules are called `docs/...` by their own repository.

With `-apply`:

- Verifies every submodule’s working tree is clean (`git status -porcelain` must be empty apart from that repository’s own `.gitignore`); rejects dirty trees immediately. The `.gitignore` exemption exists because `cgitsync` writes that one file itself in every repository holding a child, so it is routinely dirty in exactly the tree being converted; no conversion can harm it, since `git rm -cached` never touches the child’s working tree.
- Runs `git rm -cached <path>` per submodule, dropping the gitlink from the index while leaving the child’s working tree and `.git` directory intact (no re-clone, no local history lost).
- Removes the converted stanza from `.gitmodules` (deletes the file entirely when all stanzas are converted) and stages the change.
- Appends `<path>` to the parent’s `.gitignore` using the same helper that the `.gitignore` lifecycle sync uses.

After applying, review and commit the staged changes manually:

```
$ git -C REPO_ROOT status                # deleted .gitmodules, modified .
      gitignore
$ git -C REPO_ROOT commit -m "chore: retire git submodules"
```

Note: `ComplexGitSync` stores no credentials and has no private-repo authentication story. If a submodule’s upstream is private and the ambient environment lacks access, the subsequent `cgitsync initialise` or `pull` will fail at the clone step. The `import-submodules` step itself (which only touches the local index and working tree) will still succeed.

4.19 init-from-submodules

```
$ pixi run cgitsync init-from-submodules REPO_ROOT [--cgs FILE]
      [--max-depth N] [--dry-run] [--force] [--force-protocol {ssh,https
      }]
```

Adopts a submodule-based checkout in one command: `discover` the repositories under `REPO_ROOT`, write the drafted `.cgs`, `initialise` the tree in place, then convert every submodule with `import-submodules -recursive -apply`. `REPO_ROOT` is a project cloned and `git submodule update -init -recursive`’d by hand; the command performs no clone of the root itself.

The conversion must come last, and this command is the reason why. `initialise` adopts the root in place but deletes and re-clones every *other* repository straight from its remote, and those remotes still declare submodules — so a conversion run first is undone for every repository except the root. Nor can a second pass repair that: `import-submodules -recursive` follows the submodule graph declared by the root’s own `.gitmodules`, which the first pass removed, so no deeper level stays reachable. Running the two commands in the other order by hand produces a half-converted tree that looks finished.

Three conditions are checked before anything is written, so a run that cannot work fails while the checkout is still untouched:

- the discovery resolved at least one repository (skipped when `-cgs` supplies a file, which is then the authority);
- the project name matches the directory name — `CGSHOME` is resolved as `<parent>/<project-name>`, so a mismatch would send `initialise` to a sibling directory that does not exist;
- `REPO_ROOT` has a `.gitmodules`. Without one there is nothing to convert, while the clone step would still delete and re-clone every non-root repository, discarding any uncommitted work in them. `-force` overrides this one.

`-dry-run` prints the discovery and the submodules that would be converted without writing a `.cgs`, cloning, or touching any repository. `-cgs FILE` uses an existing `.cgs` instead of writing one (or writes the draft there, when `FILE` does not exist). `-max-depth N` bounds the discovery step, which is otherwise unbounded — most trees never need it. `-force-protocol` applies to the clone step only; discovery reads each repository’s configured remote as it is.

The command stops at a `READY` tree with the conversion staged but not committed — it touches every repository that held a submodule, and some of those may belong to other people — and prints the `branch/checkout/add/commit` sequence to run next.

4.20 verify

```
$ pixi run cgitssync verify [--search-dir DIR] [--repair]
```

Verifies the hash-chained `.cgitssync/lgr` register for tamper-evidence: every entry’s `prev` field must match the previous entry’s `entry_hash` (`BROKEN_LINK` if not), every entry’s own `entry_hash` must match its recomputed canonical hash (`BAD_ENTRY_HASH` if not), `seq` numbers must be gap-free and unique (`SEQ_GAP/SEQ_DUPLICATE`), and the cached `HEAD` pointer must agree with the recomputed true head (`HEAD_STALE`). A register with no entries yet — the common case, since nothing currently writes to this format — is reported clean; that is not itself a finding.

`CGSHOME` is located the same way as `status`: from `-search-dir`, else `$CGSHOME`, else the current working directory’s ancestors. Exits 0 when clean, 1 when any finding is reported.

With `-repair`, a stale `HEAD` cache is corrected to match the recomputed true head. **Entries themselves are never rewritten or deleted** by `verify`, `repair` or not — a broken chain is reported, not silently healed, since a register that can be edited back to looking clean provides no evidence of anything.

Scope note: this checks chain linkage and the `HEAD` cache only. Cross-referencing entries against the actual `state(<hash>)_n/` directories on disk (`MISSING_STATE`, `ORPHAN_STATE`, `STATE_DIGEST_MISMATCH`) is not implemented yet.

4.21 launch-release

```
$ pixi run cgitssync launch-release <name> [--gts <snapshot.gts>]
```

Checks out the frozen release tag `<name>` across the loaded `READY` tree, parent-first, and writes a fresh `.gts` snapshot. This is the release oriented CLI path for returning a workspace to a frozen version; direct tag creation remains available from the Python client API as `tag()`.

The integration suite validates local no-network lifecycle coverage through the CLI path, including initialisation and the `READY` git command cycle `add -> commit -> push -> freeze -> launch-release`.

4.22 view-tree

```
$ pixi run cgitssync view-tree [spec.cgs|snapshot.gts] [--depth N] [--collapse REPO]
```

Renders a deterministic terminal tree that focuses on repository topology and operational state. Each line reads:

```
name (node_type) [sync_state] @sha br=<branch> [fb=<fallback>]
```

`br=` is printed for *every* repository, including the common case where the branch is simply the tree’s default, so that reading `view-tree` tells you the branch each repository targets without opening the `.cgs`. `fb=` appears only when the declared fallback branch differs from that target,

since a fallback equal to the target says nothing. Use `-depth` to limit visible levels and repeat `-collapse` to hide selected subtrees.

On a tree with private mounts, `br=` is where you see pinning at work: the unprivate repositories carry the branch you moved the tree to, and each private mount still shows its own.

Branch Topology Propagation Rules (T35):

1. **Reference branch:** The root repository's current branch is the canonical reference for all repos.
2. **Leaf-to-root inheritance:** Branch targeting flows root-first via `propagate_global_branch`, verified on-disk by `ComplexGitSyncClient.validate_topology()` (Python API only) without issuing any git writes.
3. **Allowed divergence:** Repositories in a frozen (TAG) state produce a `tag_divergence` diagnostic that does *not* make the topology incoherent.
4. **Incoherent states (blocking):** `misaligned_branch` (different branch from root) and `detached_head` (detached HEAD without a tag reference).

4.23 What `cgitsync` promises a script

Exit codes. Three, meaning the same thing in every command:

Code	Means
0	The command did what was asked.
1	It ran, and the answer is no — a merge conflict, a tree that is not READY , a verification that found something.
2	It could not run — bad arguments, no workspace, a missing or unreadable file.

The distinction that matters to a script is between “I asked and the answer is no” and “I could not ask”. `validate` is the one command that judges a document rather than acting on one, so an invalid document is its answer (1); every other command exits 2 on the same document because it could not run at all.

An expected failure prints one line on `stderr` and no traceback. A traceback is therefore a defect report, not a usage error — which is the point of mapping the expected ones.

-json. `status` and `verify` accept it. Each prints one JSON object on `stdout` and nothing else; every human-facing line goes to `stderr` instead, and the exit code is unchanged, so a caller may use either signal. A failure prints one object too, carrying `"status": "error"` and the exit code, rather than leaving a pipe with nothing to parse. **-help**, **-version** and a command line that fails to parse are deliberately not JSON: the flag itself may be what failed.

The object carries `schema_version`. The promise attached to it is **additive only**: new fields may appear, existing fields do not change meaning and do not vanish.

4.24 Where a command works: the default workspace

Every command that is not given an explicit path finds its workspace from three inputs, in this order: `-search-dir`, then `$CGSHOME`, then the ancestors of the current directory. When none of them lands on a `.cgitsync` directory there is a fourth answer: an empty workspace of `ComplexGitSync`'s own, under `$HOME/.cgs` (or `$CGSPATH` when that is set).

That workspace is a real directory holding a valid `.gts` with no repositories in it. An empty tree is not an error — it is a project that has not started — so `status` answers **no living project yet**, names the three commands that start a project, and exits 0.

- **Created once, then reused.** Its path is recorded in `$HOME/.cgs/default`; later runs read that pointer instead of minting a second empty workspace.
- **It never chooses for you.** When other workspaces exist under the root they are listed, each with the `export CGSHOME=...` line that would select it, and none is selected automatically.
- **It never overrides `-search-dir`.** A directory you named that holds no workspace is an error, not an invitation to work somewhere else.

The `use_case` field. Every discovery line also says which of the two ways of running is in force: **`nested`** when the `ComplexGitSync` being executed lives inside the workspace it manages, **`standalone`** otherwise. It is observed and printed, never obeyed — nothing behaves differently because of it. It exists so that a user who believes they are in one case while standing in the other is told.

4.25 status

```
$ pixi run cgitsync status [--gts FILE]
```

Summarises tree readiness and per-repository sync state. The table reports the scope, local branch, upstream branch, local worktree state, ahead/behind tracking counts, current HEAD, and the commit recorded in the loaded `.gts`.

The `cgitsync_branch` field. The `summary` line carries `cgitsync_branch=<branch>`: the branch the project is on, which is the branch its root repository is on. It is what `cgitsync branch` and `cgitsync checkout` set, and what every other repository follows.

The table shows a branch per repository and they are not all the same on purpose — a `private/local` repository keeps this project's settings on a branch named after the project, so a project on `apoub` shows `ComplexGitSync_apoub` there. The field is the one place that says which of those branches is the project's own.

Shown	Means
<code><branch></code>	The branch the root repository is on.
<code>detached</code>	The root is parked on a commit rather than a branch. <code>cgitsync checkout <branch></code> puts the tree back.
<code>unknown</code>	There is no branch to report: no project has been loaded, or Git could not be asked.

The field is printed in every state. A field that disappears when the answer is awkward is harder to read, by eye or by script, than one that says it does not know.

The `SCOPE` column. It says, in three words, what kind of repository each row is and whether you may write to it:

Shown	Means	In the <code>.cgs</code>
<code>project</code>	this project's own; write freely	<code>no private</code>
<code>private/local</code>	shared, and yours to write	<code>private, writable</code>
<code>private/distant</code>	shared, read-only	<code>private alone</code>

`private` means the repository is shared with other projects — a configuration repository. **`local`** then means this project may write to it, normally because it sits on a branch named after

your project; **distant** means you only read it. A repository nested inside a private one is shown the same way, because it is just as shared.

commit and **push** reach the **project** rows; **-private** reaches the **private/local** rows; nothing writes to a **private/distant** row. When any private row is present, the table prints a legend saying so.

The SYNC column. It says how the branch a repository is on stands against the branch it tracks — the one named in `UPSTREAM_BRANCH`, which reads `-` when there is none. There are six values:

Shown	Means
synced	Level with the upstream.
ahead(+N)	N commits here that the remote does not have. push sends them.
behind(-N)	N commits on the remote that are not here. pull fetches them.
diverged(+N/-M)	Both, from a common ancestor. merge or pull-force resolves it.
no-upstream	This branch was never pushed, so there is nothing to compare it to. Normal for a branch you just made, and for a private/local repository that only push -private ever sends.
unknown	The branch names an upstream that does not resolve. pull or push repairs it; if it persists, the remote is unreachable or the ref was deleted.

The last two are worth telling apart. **no-upstream** is a normal state of a branch nobody has pushed yet; **unknown** is a fault. They used to share the one word **unknown**, which made the column not worth reading — an ordinary new branch looked exactly like a broken one.

unmeasured in the summary line. The summary counts the **no-upstream** and **unknown** rows as **unmeasured**, separately from **ahead** and **behind**. Those rows were previously folded into **ahead=0 behind=0**, which reported a measurement nobody had taken. A repository nobody could measure is not a repository that is level, and the summary no longer says the second when it means the first.

4.26 merge

```
$ pixi run cgitssync merge <branch> [--private] [--ff-only|--no-ff] [--dry-run] [--resolve]
```

Merges a branch across the tree, leaf-first. The argument is always the **project's** branch; each repository works out what that means for itself. A project-owned repository merges that branch. A **private/local** repository merges the branch derived from it (`<base>_<branch>`), because that is where its settings for that project branch live — it has no branch of the project's own name at all.

-private merges the writable configuration repositories instead of the project's own. Read-only ones are never merged.

A conflict in any repository leaves no repository merged.

Every repository in scope is checked before any is merged, using a read-only test that touches neither worktree nor index. A tree-wide merge that stopped halfway would leave a workspace no **.gts** describes and no command undoes, so it is prevented rather than reported. **-dry-run**

prints the plan — and, importantly, the branch each repository would actually merge, and whether it would merge at all.

The refusal names the files. When `cgitsync` refuses, it names every blocked repository and, under each, every file that conflicts:

```
merge refused; no repository was merged: ComplexGitSync: tests/unit/
  test_documents.py
```

A file that both branches changed is only listed when it genuinely conflicts. Binary files count: a tracked PDF changed on both sides conflicts just as a source file does, and is reported the same way. `-dry-run` shows the same list without merging.

-resolve: fix the conflict instead of reading about it. `-resolve` merges one repository at a time and stops at the first that conflicts, then opens that repository in your merge tool.

-resolve gives up the guarantee above.

Repositories merged before the conflict stay merged, so the tree can be left partly merged. That is the price of reaching a conflicted worktree at all, and it is why this is opt-in: the command prints the warning before it writes anything, then names what it merged, where it stopped, and which repositories it never reached.

Your own `merge.tool` is used whenever you have configured one. Otherwise VS Code is suggested when it is available, passed for that one call — your Git configuration is never written. With no merge tool available at all, the command prints the command to run by hand and stops, rather than failing.

Check out the target first. `merge` brings a branch *into* the one the tree is on, exactly as `git merge` does. Running `cgitsync merge multi-branch` while the tree is still on `multi-branch` would merge it into itself — which Git reports as success while doing nothing — so `cgitsync` refuses and says to check out the target branch first.

Landing a feature branch is therefore two commands:

```
$ pixi run cgitsync merge multi-branch
$ pixi run cgitsync merge --private multi-branch
```

4.27 pull -private

```
$ pixi run cgitsync pull --private
```

The other direction. While a project feature branch is open, work lands on a private/local repository's base branch and its derived branch does not see it. This fetches and merges the base branch into each one, so a settings branch does not drift behind the project's. A repository already on its base branch has nothing to take.

5 Document Formats

`.cgs` means **ComplexGitSync** specification, `.gts` means **GitTreeState** snapshot, and `.lgr` means **Local Git Register**.

5.1 Local Authoring Spec (.cgs)

A TOML file that describes the repository tree to manage. It is the only hand-authored input; it is never modified at runtime. The normal form has two top-level keys:

```
project = "CGS111"
repos = [
  "gitlab:CGS_test/CGS111",
  "gitlab:CGS_test/CGS112",
  "github:flipoyo/CGS1h1",
]
```

Repository identifiers have the form `provider:owner/repository`. Nested GitLab namespaces are supported; the final path segment is the repository name. The document pipeline is:

PARSE -> NORMALIZE -> VALIDATE -> canonical CgsDocument

The canonical providers and hosts are:

Provider	Host	Remote generation
github	github.com	SSH and HTTPS
gitlab	gitlab.com	SSH and HTTPS
codeberg	codeberg.org	SSH and HTTPS
custom	explicit URL required	SSH and HTTPS

Repository-ID parsing is provider-independent and deterministic. For example:

```
codeberg:GX4G/GX4G
```

normalizes to `gitprovider = codeberg`, `project_owner_name = GX4G`, and `repo_name = GX4G`. A custom identifier must use an inline table so the host is explicit:

```
{ repository = "custom:team/tool", gitprovider_url = "https://git.
  example.com" }
```

No host is inferred for `custom`. The configured access protocol then selects `git@host:owner/repository.git` or `https://host/owner/repository.git`.

The textual grammar and its only parser, `parse_repo_id()`, live in `cgs_format.py`. The provider model in `git_repo.py` receives the already-separated provider, owner, and repository fields and composes remotes; it does not parse authoring shorthand.

The `.cgs` format pipeline is deterministic and offline. A syntactically valid repository may therefore reference a host, owner, branch, or tag that is not currently reachable. Remote existence and reference availability are checked only by explicit Git operations during runtime resolution and execution.

Normalization supplies `default_branch = main` (see “Branches in a `.cgs`” below for the full chain), makes `fallback_branch` equal to that default, selects `access_protocol = ssh`, enables `nested_config = auto`, and uses the repository name as `relative_path`. If exactly one repository name matches `project`, its path is inferred as `..`.

Advanced inline tables are used only where a value differs from those defaults:

```
project = "demo"
repos = [
  "github:owner/demo",
  { repository = "github:owner/docs", relative_path = "documentation",
    access_protocol = "https" },
]
```

The legacy advanced `[project]` and `[[repos]]` tables remain accepted for project-wide branch settings, custom provider URLs, tags, and other explicit overrides. Unknown providers, malformed identifiers, duplicates, and missing `project` or `repos` are rejected before a `GitTree` is built.

Two checked-in files document this boundary. Copy `examples/template.cgs` for normal authoring. Its equivalent `examples/normalized_template.cgs` expands every inferred field and is provided as a developer reference for the canonical `CgsDocument` shape. The two files are regression-tested for semantic equivalence.

The repository also keeps three checked-in, didactically ordered tutorials under `tutorials/` — easiest first, see `tutorials/README.md` for the guided order — exercising this authoring surface end to end: `tutorials/01_first_multi_repo_workspace.md` walks a fully exercised local sandbox topology (CGS11) through the complete CLI lifecycle; `tutorials/02_onboarding_a_real_build_tree.md` hand-authors a `.cgs` for the larger, real-world `cawaqs` tree and the point where `cgitsync` hands off to the project’s existing make-based build workflow; and `tutorials/03_adopting_a_real_project.md` covers the real-world `cawaqsviz` project (not synthetic), which has no `.cgs` of its own and still tracks three repositories on two levels as git submodules, as one verified procedure: clone, check out the submodules, adopt the whole tree with `init-from-submodules` (`discover + initialise + import-submodules`, in that order), then `branch/checkout/add/commit/push`.

`examples/cawaqsviz.cgs`, the hand-authored alternative mentioned in `tutorials/03_adopting_a_real_project.md` above, describes <https://gitlab.com/cawaqs/gviz/cawaqsviz> and exercises several patterns above at once: a nested GitLab subgroup path (`cawaqs/gviz/cawaqsviz`, three segments rather than a plain owner/repository pair), an explicit `relative_path = "."` on the root entry rather than relying on the project-name auto-mount rule, two GitHub children mounted at non-default relative paths (`external/HydrologicalTwinAlphaSeries` and `docs/CWV_user_guide`). Neither child has a `.cgs` of its own, but no override is needed for that: the default `auto` nested discovery resolves a repository with zero `*.cgs` matches as a normal leaf. This file previously shipped with a nonexistent repository identifier and an invalid `default_branch`; it was corrected and given real end-to-end test coverage (`tests/integration/test_cgsi_topology.py`, and one verified run against the live repositories). See `tutorials/03_adopting_a_real_project.md` for the full worked example, and `bootstrap` above for running `ComplexGitSync` against it without cloning `ComplexGitSync` inside the tree it manages.

Generated specifications use the same concise representation. The `GitTree.to_cgs()` convenience method delegates model projection to `cgs_format.py`; neither `GitTree` nor `GitRepo` formats TOML or implements the authoring grammar. Serialization omits every default that normalization can deterministically reconstruct and retains inline tables only for exceptional settings such as a non-default branch, path, protocol, tag, custom host, or discovery rule.

The supported round trip is semantic:

```
.cgs -> CgsDocument -> GitTree -> CgsDocument -> .cgs
```

Parsing and normalizing the generated file must reproduce the initial canonical document. Whitespace, comments, and TOML table layout need not be byte-for-byte identical.

5.2 Branches in a `.cgs`

Four fields decide which branch a repository lands on. Three of them fall back to each other, so a `.cgs` that names no branch at all still sends every repository somewhere.

Field	Where it is written	Default
<code>project.default_branch</code>	the project table	<code>main</code>
<code>repos[].default_branch</code>	one repository entry	the project’s
<code>repos[].fallback_branch</code>	one repository entry	that repository’s own
<code>repos[].private</code>	one repository entry	<code>false</code>
<code>repos[].writable</code>	one repository entry	<code>false</code>

Read the first three as one chain, longest reach first:

```
fallback_branch -> default_branch -> project.default_branch -> main
```

`project.default_branch` is the tree's branch: every repository inherits it unless the repository says otherwise. `repos[].default_branch` is the branch that one repository targets. `repos[].fallback_branch` is used when the target branch does not exist on the remote — `cgitsync` clones the fallback instead and records that it did so, rather than failing.

Each link is silent. Nothing warns you that a decision was made, so **state `project.default_branch` in every `.cgs` you write**, even when the answer is `main`. A reader of the file can then see the branch the tree lands on without knowing what `cgitsync`'s built-in default happens to be. Every `.cgs` shipped with `ComplexGitSync` does this, and a test enforces it.

5.2.1 What private means

A repository is **private** when it is **shared with other projects** and must stay on its own branch.

Branch moves stop at a private repository. Tags do not.

`cgitsync branch <name>` and `cgitsync checkout <name>` skip a private repository and leave it on its declared `default_branch`, because moving it would drag every other project that mounts it onto your branch.

5.2.2 Read-only and writable configuration repositories

A private repository is a **configuration repository**: shared with other projects rather than owned by this one. `private = true` alone means **read-only** — `cgitsync` will not write to it. Add `writable = true` for the case where the repository lives outside your project but the part you use is yours, normally because it sits on a branch named after your project that nobody else reads:

```
{ repository = "github:you/.sharedSpec", default_branch = "main",  
  private = true },  
{ repository = "github:you/.myNotes", default_branch = "MyProject",  
  private = true, writable = true },
```

Read-only is the default because the expensive mistake is writing to something shared by accident, never the reverse. `writable = true` on an unprivate repository is rejected at validation: an unprivate repository is this project's own and is always writable.

This is a **safety rail, not a permission system**. It stops a tree-wide sweep from writing where you did not mean; it cannot stop anyone running `git` in that directory by hand, and it is no substitute for branch protection on the remote.

A private/local repository gets a branch per project branch. A private/distant repository is private to its owner: you read it, only that owner writes to it, and nothing you do moves it — so a branch move leaves it exactly where it is. A private/local repository is the opposite case: it holds settings that configure *your* project and you commit to it, so it needs somewhere to record them per project branch.

`<project name>` on `main`, `<project name>_<branch>` on any other branch

The base is the ****project's**** name. `main` takes no suffix, because the project's main line's settings branch is simply the project's name — which is what every existing tree already has,

so nothing has to move. The separator is an underscore because hyphens already occur inside branch names.

A user never types the derived name. `cgitsync checkout <branch>` puts the project's own repositories on that branch and each private/local one on the branch this rule names, creating it when it is missing.

Without this, one settings branch serves every branch of the project, and documentation written for an unfinished feature is live on the default branch describing code that is not there.

-private everywhere. Every command that touches Git takes it: `clone` aside, that is `pull`, `pull-force`, `checkout`, `branch`, `add`, `rm`, `commit`, `push`, `merge`, `tag` and `freeze`. It narrows the command to the writable configuration repositories alone, so they can be worked on without reaching for plain `git`. Read-only configuration repositories are never written to, with or without it. Four of those commands also take `-all`, described next.

-all: both halves, one command. Running every write twice — once plainly, once with `-private` — is the right default when the two changes are different changes. When they are one change, `-all` does both halves in a single pass, under a single commit message. Four commands take it: `add`, `commit`, `push` and `merge`.

Form	Reaches
<code>cgitsync <cmd></code>	this project's own repositories
<code>cgitsync <cmd> -private</code>	the writable configuration repositories
<code>cgitsync <cmd> -all</code>	both of the above, in one pass

The plain form has not moved, so nothing an existing script runs changes meaning. `-all` and `-private` cannot be combined: `-all` already includes everything `-private` selects. No form of either writes to a read-only configuration repository — “all” means every repository `cgitsync` may write to, never every repository in the tree.

The output still names the two halves apart, so giving up the typing does not mean giving up seeing which repository was written as what. On a tree with no writable configuration repository, `-all` does the project half and reports the other as empty; that is the common case, not a fault. An explicit `-private` on the same tree still stops, because there the user asked for something that is not there.

`merge -all` checks every repository before merging any.

That is why `-all` is one pass and never two. Two sequential merges mean two preflights: the project half would merge, the configuration half would then refuse, and the tree would be left half-merged — exactly the state `merge`'s whole-scope check exists to prevent. `-dry-run` previews the same combined scope, and `-resolve` keeps its own warning and its deliberate partial progress.

A pin covers everything inside it. A configuration repository often holds repositories of its own, reached through its nested `.cgs`. Those are just as shared as the repository holding them, so they inherit its state: **a repository inside a read-only configuration repository is read-only, even when its own entry says nothing about it.** One inside a writable configuration repository is writable, and `-private` reaches it.

A nested entry may restrict itself further — writing `private = true` on its own line, with no `writable`, makes it read-only inside a writable parent. It cannot go the other way: `writable = true` inside a read-only configuration repository is ignored, because a repository can never be more open than the one that contains it.

5.2.3 Which repositories each command touches

Command	Reaches	Why
add, commit, push	this project's own	they write this project's history
the same, -private	writable config repos	shared, but on your own branch
branch, checkout	this project's own	a branch must not spread
tag, freeze-release	everything writable	they create <i>and push</i> a tag
clone, pull, status	everything	reading a shared repo is the point

Preflight follows the same scope. Each command checks only the repositories it is about to touch. A configuration repository that is behind its upstream, or sitting on its own branch, never blocks a `commit` of this project's own work — that repository is not part of the commit. Under `-private` it is in scope, and its branch is then checked against the branch it is *private* to, not against the one the project is on.

`-private` is exclusive, not additive: it selects *only* the writable configuration repositories, so a shared repository gets its own command and its own commit message rather than being swept into this project's. Asking for it in a tree with no writable configuration repository is an error naming the read-only ones, not a silent no-op. Every scoped command reports what it left alone:

```
scope=project skipped=5 configuration repo(s) (.claude, .localSpec with
--private)
```

`tag` and `freeze-release` stop at read-only repositories because they push. Nothing is lost: a `.gts` snapshot records every repository's exact `commit_sha`, read-only ones included, so an exact tree is rebuilt from the snapshot rather than from tags.

In practice shared config mounts are usually dot-named (`.agentSpec`, `.localSpec`), and `cgitsync discover` drafts a dot-named repository with `private = true` for that reason. The rule is *shared*, not *hidden*, though: a plainly named repository that two projects mount should be private too.

The branch a repository is private to decides who sees your commits. A mount private to a branch named after your own project is yours to push freely. A mount private to `main` is read by every project that mounts it, so a commit there is published the moment it is pushed — no branch, no pull request, no review. Check which of the two you are looking at before committing to a private mount.

5.2.4 The two meanings of “force”

`pull-force` is destructive: it discards local changes. `-force-protocol` is not: it only switches a remote between `ssh` and `https`. The two are unrelated despite the shared word.

5.3 Repository Placement and Nested Discovery

For every non-root entry, `relative_path` is relative to the parent repository represented by the current `.cgs`; it is not relative to the process working directory. At the top level that parent is the project root. While expanding a nested config, it is the repository containing that config. The default is the repository name. A unique repository matching the project name uses `.` because it represents the current root.

The `nested_config` option controls whether traversal continues after the repository is available:

- `auto` (default) loads the sole root-level `*.cgs` file, if one exists; finding none resolves the repository as a normal leaf; multiple matches are rejected as ambiguous.

A consequence worth knowing before you publish a repository: **a repository that other people may mount inside their tree must keep exactly one .cgs at its root**, because `auto` has no way to choose between two. The tree's own root is exempt — it is never scanned for its own `.cgs`, since that would recurse — so a root holding two `.cgs` files works fine until the day someone mounts it. The error names both files and how to resolve it.

- `disabled` does not inspect the repository for nested config.
- A relative path ending in `.cgs` loads that exact file inside the repository, failing if it does not exist. It may not escape the repository root.

For the integration topology, `CGSih2.cgs` contains:

```
{ repository = "github:flipoyo/CGSih1", relative_path = "../CGSih1" }
```

Because the config describes the tree below `CGSih2`, the path resolves from that directory:

```
<CGSHOME>/CGSih2/../CGSih1 -> <CGSHOME>/CGSih1
```

It therefore references the existing sibling declared by `CGSih1.cgs`, instead of cloning another `CGSih1` under `CGSih2`. Discovery recognizes the already-registered absolute path and keeps the canonical entry — this guard runs before `auto` would ever get a chance to reopen `CGSih1.cgs` through the cross-reference, so no `nested_config` override is needed here.

5.4 Git Tree State Snapshot (.gts)

A TOML file generated automatically by bootstrapping and release operations. It must never be hand-edited. It captures the exact commit SHAs and lifecycle state of the tree at a point in time.

Top-level sections:

- `[document]` — `format_version`, `schema_version`, `generated_at`, `command_origin`, `hash_algorithm`, `snapshot_hash`.
- `[project]` — project name and `root_absolute_path`.
- `[tree_state]` — `lifecycle_state`, `is_ready`, `registry_complete`.
- `[[repo_state]]` — one entry per repo; includes `commit_sha`, `sync_state`, and ref information.

Why both version fields and hash metadata exist:

- `format_version` identifies the broad document family and keeps long-range compatibility intent explicit.
- `schema_version` identifies the exact field-level contract used by validation and canonical snapshot serialization.
- `snapshot_hash` (with `hash_algorithm = "sha256"`) provides deterministic workspace identity: identical tree state produces identical digest even if volatile metadata (`generated_at`, `command_origin`) differs.

How this is used at runtime:

- During snapshot generation, `ComplexGitSync` writes `schema_version`, `hash_algorithm`, and canonical `snapshot_hash`.

- During load/validation, `snapshot_hash` is recomputed from the canonical payload and must match.
- The project-local `.lgr` register uses this canonical hash to deduplicate equivalent snapshots.

5.5 Local Git Register and Sync Ledger (`.lgr`)

A TOML file maintained per project at `<project-root>/<Project_name>.lgr`. It is updated automatically whenever a `.gts` snapshot is written.

Sections:

- `[register]` — current snapshot pointer (`current_snapshot_id`, `current_snapshot_hash`, `current_snapshot_path`).
- `[[snapshots]]` — list of stable `gts-XXXXXX` identifiers, one per unique workspace state, deduplicated by SHA-256 hash.
- `[[ledger]]` — append-only list of synchronisation events forming a DAG via `parent_sync_ids`.

Ledger event schema:

```
[[ledger]]
sync_id      = "lgr-000001"
parent_sync_ids = []
operation    = "clone"
timestamp    = "2026-05-20T19:48:50.159Z"
actor       = "username"
workspace_hash = "5f7c8a2d..." % 64-character SHA-256 hex digest
gts_snapshot_id = "gts-000001"
affected_repos = ["demo", "dep-a"]
```

The `workspace_hash` field equals the canonical SHA-256 digest (`document.snapshot_hash`) of the associated `.gts` file, creating a direct link between each ledger event and the immutable workspace state it records. Events are parents-before-children in DAG order.

6 Worked Examples: From `CGSil1` to `cawaqsviz`

`ComplexGitSync` ships a same-numbered tutorial under `tutorials/` for each level below (`tutorials/README.md` lists them in order). Run through them in order: each one adds exactly one new idea on top of the last, so together they form a guided path from the simplest possible `.cgs` to a real project with no `.cgs` of its own at all.

Levels 2 and 3 have their `.cgs` checked in under `examples/`. Level 1 does not: `CGSil1.cgs` lives in the `CGSil1` repository itself, which is what the transcript below clones.

Every command below is a Pixi task. `cgitsync` is not a globally installed executable — always run `pixi run cgitsync ...`, never a bare `cgitsync ...` (see the §1 “CLI Overview” note above).

Level	Topology	Entry point	New idea introduced
1	CGSil1	hand-authored .cgs	minimal spec, name-matching auto-mount
2	cawaqs	hand-authored .cgs	the same minimal-field style as Level 1, at 19-repo scale
3a	cawaqsviz	hand-authored .cgs	explicit overrides on a real, irregular tree
3b	cawaqsviz	discover	drafting a .cgs from a checkout, no authoring at all
3c	cawaqsviz	import-submodules	migrating an existing git-submodules project

Levels 3a–3c are not three different projects: they are three different *starting points* for the same real project, `cawaqsviz`, chosen so this section can demonstrate every route `ComplexGitSync` offers for arriving at a working `.cgs` — write it by hand, derive it from a checkout, or migrate it from an existing submodules setup — without inventing a fourth toy topology to do it. It is the most advanced level not because `cawaqsviz` is a larger tree than `cawaqs` (it is much smaller, three repos against nineteen) but because, unlike either earlier level, it has no `.cgs` anywhere upstream to copy from.

6.1 Level 1 — CGSil1: the minimal synthetic topology

`CGSil1` is the easiest way into `ComplexGitSync`: a 4-repository, mixed-provider tree with every field left at its default. Its full `.cgs` and the rationale for each line are in §1 “Local Authoring Spec” above; `tutorials/01_first_multi_repo_workspace.md` carries the full transcript with expected output for every step. What follows is the command sequence itself.

```
# 1. Fetch the root repo and cgitsync itself (nested mode: cgitsync is
#    cloned inside the tree it will manage).
$ git clone https://gitlab.com/CGS_test/CGSil1
$ cd CGSil1 && git clone https://github.com/flipoyo/ComplexGitSync
$ cd ComplexGitSync

# 2. Check the spec offline, no network beyond parsing.
$ pixi run cgitsync validate ../CGSil1.cgs

# 3. Preview the topology before touching disk.
$ pixi run cgitsync view-tree ../CGSil1.cgs

# 4. Clone every declared repo and reach READY.
$ pixi run cgitsync initialise ../CGSil1.cgs

# 5. The ordinary git cycle, run tree-wide from here on --- no path
#    argument needed, the .gts snapshot is discovered automatically.
$ pixi run cgitsync pull
$ pixi run cgitsync add
$ pixi run cgitsync commit "my commit message"
$ pixi run cgitsync push

# 6. Minimalist release: stage, commit, pull, push, freeze in one call.
$ pixi run cgitsync freeze-release v1.1.0 "release v1.1.0"

# 7. Return to a frozen release at any later point.
$ pixi run cgitsync launch-release v1.1.0
```

The only `.cgs` idea this level relies on is the **name-matching auto-mount**: because `CGSil1`’s repository identifier’s last segment equals `project = "CGSil1"`, the root is inferred

at `relative_path = "."` with no override written anywhere. That convenience is exactly what stops being safe once a project’s identifier and display name diverge — which is where Level 3a picks up.

6.2 Level 2 — cawaqs: the same minimal style, at scale

`cawaqs` scales Level 1’s minimal, no-override authoring style up to a real 19-repository build tree (root + 17 physics libraries across five GitLab groups + one required build-tooling repo), instead of a synthetic 4-repo one. Every entry’s on-disk layout already matches the default `relative_path = <repo_name>`, so not a single `relative_path` override is written anywhere in `examples/cawaqs.cgs` — only the shared branch-fallback convention. Full narrative in `tutorials/02_onboarding_a_real_build_tree.md`.

```
$ pixi run cgitssync validate examples/cawaqs.cgs
$ pixi run cgitssync bootstrap examples/cawaqs.cgs cawaqs-smoke-test
$ pixi run cgitssync view-tree --search-dir <path bootstrap printed>

# Move the whole 19-repo tree to a shared branch in one call, falling
# back to main per-repo wherever that branch does not exist.
$ pixi run cgitssync checkout my-feature-branch
```

Reaching READY here only replaces the *repository-fetching and branch-selection* half of `cawaqs`’s own `make_Cawaqs.sh/make_Cawaqs_from_branches.sh` scripts. `ComplexGitSync` does not compile anything: the remaining `make -f Makefile` build step is still the user’s job, and only succeeds from this tree with `LIB_HYDROSYSTEM_PATH` left unset or pointed at the bootstrapped root — the alternative shared, decoupled install layout those scripts also support is out of scope for `ComplexGitSync`’s containment model.

`discover` (Level 3b below) is not a route into `cawaqs.cgs`: the 17 library repositories never coexist inside one directory tree until *after* a `.cgs` already lists them, so there is no single checkout for a filesystem walk to scan. Authoring this one from documented developer knowledge, the way Level 3a does for `cawaqsviz`, is the only available route.

6.3 Level 3 — cawaqsviz: three ways to the same .cgs

`cawaqsviz` (<https://gitlab.com/cawaqs/gviz/cawaqsviz>) is a real GitLab project, nested three path segments deep, with two GitHub children that were, until recently, plain git submodules. One of those children, `HydrologicalTwinAlphaSeries`, holds a submodule of its own at `docs/hydrological_twin`: four repositories on two levels, which is why every command below uses `-recursive`. It has no `.cgs` of its own anywhere upstream. That makes it a good vehicle for showing all three ways `ComplexGitSync` can produce a working `.cgs` for a project it does not already control the source of, ordered here by how much the operator has to already know about the tree before running the command. Full narrative, as one combined ten-step procedure rather than three independent routes, in `tutorials/03_adopting_a_real_project.md`.

6.3.1 3a — Write it by hand

The most explicit, and most error-prone, route: read the project’s own `.gitmodules` and topology, then author `examples/cawaqsviz.cgs` directly. Its corrected content and the exact mistakes the first draft made (wrong nested-subgroup identifier, missing `relative_path` overrides) are already spelled out in §1 “Local Authoring Spec”; this level is about running it, not re-deriving it:

```
$ pixi run cgitssync validate examples/cawaqsviz.cgs
$ pixi run cgitssync bootstrap examples/cawaqsviz.cgs cawaqsviz-demo
$ pixi run cgitssync view-tree --search-dir <path bootstrap printed>
```

`bootstrap` is used instead of `initialise` here because, unlike `CGSi11` above, this example is not meant to have `ComplexGitSync` cloned inside the tree it manages — see `bootstrap` in §1 for why the two commands pick different default locations.

6.3.2 3b — Derive it with `discover` (autodiscover mode)

The opposite route: start from nothing but a checkout, and let `ComplexGitSync` read the filesystem instead of writing the `.cgs` by hand. This is the route to prefer whenever a checkout is already available, since it cannot repeat the hand-authoring mistakes of 3a — `relative_path` falls out of the walk directly instead of being typed.

```
# A plain clone leaves submodule paths empty; initialise them first so
# discover has something to find at those paths. --recursive reaches the
# second level too: without it, hydrological_twin stays an empty
# directory and discover cannot see a repository that is not there.
$ git clone https://gitlab.com/cawaqsviz/gviz/cawaqsviz cawaqsviz
$ cd cawaqsviz && git submodule update --init --recursive

# Dry run: report what discover sees, write nothing.
$ pixi run cgitsync discover . --max-depth 3

# Satisfied with the report: draft the .cgs.
$ pixi run cgitsync discover . --write cawaqsviz-discovered.cgs
$ pixi run cgitsync validate cawaqsviz-discovered.cgs
```

The draft reconstructs 3a's hand-corrected file field-for-field: the root at `relative_path = "."` and every other repository at its real submodule path (`external/HydrologicalTwinAlphaSeries`, `docs/CWV_user_guide`, `external/HydrologicalTwinAlphaSeries/docs/hydrological_twin`) rather than their bare repo names. That last path lies inside `HydrologicalTwinAlphaSeries`, so the report marks it `inside`: `external/HydrologicalTwinAlphaSeries` and the file is read back with `HydrologicalTwinAlphaSeries` as its parent. No repository here has a `.cgs` of its own, and `discover` writes no `nested_config` override for that: the default "auto" already resolves a repository with zero nested `*.cgs` matches cleanly. See `discover` in §1 for exactly what is and is not reported (no network calls, unresolvable remotes are warned about and left out, never guessed at).

6.3.3 3c — Migrate it with `import-submodules`

The historical route: before either `.cgs` above existed, `cawaqsviz` tracked its children as real git submodules. This mode starts from that state and converts it, rather than describing a tree that is assumed to already be plain clones.

```
# Dry run against a real cawaqsviz checkout that still has submodules:
# reports name, path, URL, branch and declaring .gitmodules per
# submodule;
# changes nothing. --recursive reaches HydrologicalTwinAlphaSeries' own
# submodule; without it, only the root's two are converted.
$ pixi run cgitsync import-submodules /path/to/cawaqsviz --recursive

# Convert: drop the gitlinks, retire .gitmodules, extend .gitignore.
$ pixi run cgitsync import-submodules /path/to/cawaqsviz --recursive --
  apply

# Review and commit the resulting working-tree changes as usual. Two
# repositories have changes here, one per converted level.
$ cd /path/to/cawaqsviz && git status
```

```
$ git commit -m "chore: retire git submodules in favour of
  ComplexGitSync"
```

This is the whole job: turning gitlinks into plain clones on disk. It does not also write a `.cgs` — `.gitmodules` never records the root’s own identity, and a checkout worth converting already has a `.cgs` (3a) or can get one from `discover` (3b), run before or after `-apply`. The full before/after picture (index, `.gitmodules`, `.gitignore`) and the automated test that proves it against fixture repositories are in `tutorials/03_adopting_a_real_project.md` §4 — this level is the condensed command sequence, not a restatement of that walkthrough. `import-submodules` and `discover` answer different questions about the same tree: `discover` reports what is *checked out*, `import-submodules` reads what git *declares* in `.gitmodules` and acts on it — a submodule path that was never initialised is invisible to 3b but not to 3c.

6.3.4 A shared caveat across all three

`ComplexGitSync` stores no credentials and has no private-repository authentication story (§1 “`import-submodules`” and “`discover`”). All three routes above assume every repository in the tree is reachable anonymously or through credentials the ambient environment (`ssh-agent`, an HTTPS credential helper) already holds.

7 Configuration and Environment

`ComplexGitSync` uses project files for topology and does not require global topology configuration. Snapshot discovery can use `CGSHOME`; when it is not set, the `initialise(.cgs)` output-path default is `CGSPATH=../..` (nested mode: `ComplexGitSync` cloned inside the tree it manages); later commands can also walk upward from the current directory to find `.cgitsync/state/.bootstrap(.cgs)` uses a different default instead — `CGSPATH=$HOME/.cgs/CGS<timestamp>/`, created automatically — since it is meant for a `ComplexGitSync` clone that is not nested inside any project tree; see `bootstrap` above. The logging and runtime-state subsystems use the user state directory by default:

- logs: `/.local/state/ComplexGitSync/logs/`
- latest runtime-state mapping: `/.local/state/ComplexGitSync/runtime/`

If `XDG_STATE_HOME` is set, these directories are rooted there instead.

8 Error Reference

ConfigValidationError Raised when `project` or `repos` is missing, a repository identifier is malformed, or a canonical field is invalid (for example an unknown `gitprovider`).

GitSyncError Raised when clone-time Git operations fail, for example because the remote branch cannot be resolved or the target directory is already occupied.

Exit code 1 General CLI failure (parse error, validation error).

Exit code 2 Command exists but is not yet implemented.

9 Troubleshooting

“gitprovider_url is required for custom provider” Add a `gitprovider_url` field to the offending `[[repos]]` entry in your `.cgs` file.

validate exits non-zero despite a correct-looking file Check that `project` is present and every `repos` entry uses `provider:owner/repository`. For advanced `[[repos]]` blocks without a `repository` shorthand, include `project_owner_name` and `project_name`.

Clone destination already exists and is not empty Re-run `clone` with `-target-dir` to force a clean location, or let `cgitsync` choose a suffixed default directory.

No cloneable branch found for <repo> Confirm that the remote exposes either the repo's `default_branch` or its `fallback_branch`.

PyYAML import error when calling `from_yaml/to_yaml` Add PyYAML in the Pixi environment: `pixi add pyyaml`.